

30 pièges SQL

junior vs confirmé

Ce qui sépare une requête qui tourne d'une requête qui dit vrai

Ce n'est pas la syntaxe SQL qui élimine un junior. Ce sont les pièges de raisonnement. Une jointure qui double tes lignes, un `WHERE` qui filtre avant un `GROUP BY` que tu croyais après, un `NULL` qui casse ton `COUNT` sans rien dire. Le junior écrit une requête qui « marche ». Le data analyst confirmé sait pourquoi elle peut mentir. Voici 30 pièges qui reviennent en test technique et en production. Pour chacun : le piège exact, le symptôme, la conséquence sur tes chiffres, et la correction d'un profil confirmé. Garde le guide. Le jour de ton test, tu sauras où regarder.

■ ■ ■ Jointures : là où tes lignes se dupliquent ou disparaissent

1. Le `LEFT JOIN` avec une condition sur la table de droite dans le `WHERE`

Le piège : Tu écris `LEFT JOIN commandes c ... WHERE c.statut = 'payee'`. Tu crois garder tous les clients.

Symptôme : Les clients sans commande payée disparaissent du résultat. Ton `LEFT JOIN` se comporte comme un `INNER JOIN`.

Conséquence : Le filtre s'applique après la jointure, donc il rejette les lignes à `NULL` venues de la table de droite. Ton nombre de clients chute sans erreur, et personne ne le voit.

Correction : Si tu veux préserver la gauche, mets la condition dans le `ON` (filtre avant la jointure). Ne garde dans le `WHERE` que les conditions sur la table de gauche.

```
LEFT JOIN commandes c ON c.client_id = cl.id
                        AND c.statut = 'payee';
```

2. Le `JOIN` sur une clé non unique côté droit

Le piège : Tu joins `clients` sur `commandes` et tu sommes un montant, en supposant qu'une ligne reste une ligne.

Symptôme : Le nombre de lignes augmente. Si un client a 3 commandes, sa ligne est triplée.

Conséquence : C'est un produit cartésien partiel. C'est la cause numéro 1 des chiffres d'affaires gonflés dans les dashboards : une donnée de référence se retrouve comptée autant de fois qu'il y a de correspondances.

Correction : Vérifie l'unicité de la clé côté droit avant de joindre. Si la table de droite a plusieurs lignes par clé, agrège-la d'abord dans une sous-requête ou une CTE, puis joins sur le résultat déjà au bon grain.

3. Croire qu'un `INNER JOIN` ne perd jamais de lignes

Le piège : Tu enchaînes des `INNER JOIN` sur 4 tables sans y penser.

Symptôme : Le total final est plus bas que prévu, sans message.

Conséquence : Chaque `INNER JOIN` jette silencieusement les lignes sans correspondance. Sur une chaîne de jointures, les pertes se cumulent et ton périmètre rétrécit à chaque étape.

Correction : Compte tes lignes avant et après chaque jointure. Si tu dois garder le périmètre de la table principale, utilise un `LEFT JOIN` et traite les `NULL` ensuite, plutôt qu'un `INNER JOIN` qui ampute en silence.

4. Chercher « les A sans B » avec `NOT IN`

Le piège : `WHERE id NOT IN (SELECT client_id FROM commandes)` pour trouver les clients sans commande.

Symptôme : Si la sous-requête renvoie un seul `NULL`, le résultat est vide.

Conséquence : En logique ternaire SQL, `id != NULL` vaut `UNKNOWN`, et le `NOT IN` ne renvoie alors plus rien de fiable. Ton contrôle d'intégrité passe pour « tout est ok » alors qu'il est cassé.

Correction : Utilise un anti-join robuste : `LEFT JOIN` suivi de `WHERE b.id IS NULL`, ou `NOT EXISTS`. Les deux gèrent correctement les `NULL`.

5. Oublier le « spine » de dates et de dimensions

Le piège : Tu agrèges les ventes par mois directement depuis la table des transactions.

Symptôme : Les mois sans vente n'apparaissent pas. Ta courbe a des trous.

Conséquence : Un graphique chronologique avec des mois manquants fausse la lecture : une rupture à zéro ressemble à une absence de donnée, ou pire, est interpolée comme si de rien n'était.

Correction : Crée un squelette (un `CROSS JOIN` entre tous les mois et toutes les dimensions voulues), puis `LEFT JOIN` tes données dessus et `COALESCE` à 0. Tu garantis une ligne par période, même vide.

■ ■ ■ Agrégation et `GROUP BY` : compter juste

6. Confondre `COUNT(*)`, `COUNT(col)` et `COUNT(DISTINCT col)`

Le piège : Tu écris `COUNT(*)` pour « le nombre de clients » alors que la table contient des lignes en doublon ou des colonnes à `NULL`.

Symptôme : Le chiffre ne colle pas avec le métier, sans erreur SQL.

Conséquence : COUNT(*) compte toutes les lignes (y compris les NULL), COUNT(col) ignore les NULL de la colonne, COUNT(DISTINCT col) compte les valeurs uniques non NULL. Te tromper de variante fausse un KPI sans alerte.

Correction : Choisis la variante selon la question métier exacte. « Combien de clients distincts ont commandé » ? C'est COUNT(DISTINCT client_id), jamais COUNT(*).

7. Filtrer sur un alias du SELECT dans le WHERE

Le piège : SELECT prix*qte AS ca ... WHERE ca > 1000.

Symptôme : Erreur « column ca does not exist » (ou équivalent selon le moteur).

Conséquence : Dans l'ordre logique SQL, le WHERE est évalué avant le SELECT : l'alias n'existe pas encore. Aucun moteur majeur (PostgreSQL, MySQL, SQL Server, BigQuery) ne l'autorise dans le WHERE.

Correction : Répète l'expression dans le WHERE, ou passe par une CTE et filtre dans la requête externe. L'alias reste utilisable dans ORDER BY, et parfois dans GROUP BY/HAVING selon le moteur.

8. Mettre une condition d'agrégat dans le WHERE au lieu du HAVING

Le piège : WHERE SUM(montant) > 1000 GROUP BY client_id.

Symptôme : Erreur « aggregate functions are not allowed in WHERE ».

Conséquence : Le WHERE filtre des lignes avant le regroupement, donc il ne peut pas voir un agrégat qui n'existe pas encore. Le HAVING filtre après le GROUP BY, sur les groupes.

Correction : WHERE pour filtrer les lignes en amont (gain de perf), HAVING pour filtrer les groupes après agrégation. Les deux sont complémentaires, pas interchangeables.

9. GROUP BY par position ordinale en production

Le piège : GROUP BY 1, 2 pour aller vite.

Symptôme : Tout marche, jusqu'au jour où quelqu'un réordonne les colonnes du SELECT.

Conséquence : Le regroupement se fait alors sur les mauvaises colonnes, sans aucune erreur de syntaxe. Le bug est silencieux et peut vivre des mois dans un reporting.

Correction : Nomme toujours les colonnes explicitement dans le GROUP BY. La position ordinale est tolérable pour une exploration jetable, jamais pour une requête qui vit.

10. Sommer un montant déjà dupliqué par une jointure

Le piège : Tu joins une table d'en-têtes de commande à ses lignes de détail, puis tu fais SUM(commande.montant_total).

Symptôme : Le chiffre d'affaires est gonflé d'un facteur égal au nombre de lignes de détail par commande.

Conséquence : La jointure a répété le montant d'en-tête sur chaque ligne de détail. Tu sommes le même montant plusieurs fois : l'erreur d'agrégation par excellence.

Correction : Agrège chaque table à son grain avant de joindre, ou somme une mesure additive au bon niveau. En dernier recours, `SUM(DISTINCT ...)` n'est presque jamais la bonne réponse : corrige le grain, pas le symptôme.

■ ■ ■ NULL : la valeur qui ne se compare pas

11. Comparer un NULL avec = ou !=

Le piège : `WHERE statut != 'actif'` en pensant récupérer aussi les lignes où statut est NULL.

Symptôme : Les lignes à NULL sont absentes du résultat.

Conséquence : `NULL != 'actif'` vaut UNKNOWN, pas TRUE, et le WHERE ne garde que ce qui est strictement TRUE. Tu perds toute une catégorie de lignes sans t'en rendre compte.

Correction : Teste l'absence avec `IS NULL / IS NOT NULL`. Pour inclure les NULL : `WHERE statut != 'actif' OR statut IS NULL`.

12. Attendre 0 d'un SUM sur des valeurs toutes NULL

Le piège : `SUM(montant)` sur un groupe où aucune valeur n'est renseignée.

Symptôme : La cellule affiche NULL, pas 0.

Conséquence : Un NULL qui se propage dans un calcul aval (division, ratio, addition) contamine tout. Un total « vide » qui devrait être 0 casse une formule ou un graphique.

Correction : Encapsule avec `COALESCE(SUM(montant), 0)` quand le métier attend un 0. À l'inverse, AVG ignore les NULL au dénominateur : sache si tu veux une moyenne sur les présents ou sur tout l'effectif.

13. La concaténation contaminée par un NULL

Le piège : `prenom || ' ' || nom` avec un nom parfois NULL.

Symptôme : La chaîne entière devient NULL, pas juste le prénom seul.

Conséquence : L'opérateur `||` propage le NULL. Des libellés entiers disparaissent dans un export ou une étiquette. Attention : `CONCAT()` varie selon le moteur (PostgreSQL et SQL Server ignorent les NULL, MySQL renvoie NULL si un argument l'est).

Correction : `COALESCE` chaque morceau avant concaténation, ou utilise `CONCAT_WS` qui gère proprement le séparateur et les NULL.

14. Confondre chaîne vide et NULL

Le piège : Tu nettoies une colonne avec `WHERE email IS NOT NULL` en oubliant les chaînes vides `' '`.

Symptôme : Des « emails » vides passent le filtre et polluent les comptages.

Conséquence : Une chaîne vide n'est pas un `NULL` : elle est non nulle pour SQL. Sur Oracle, à l'inverse, une chaîne vide est traitée comme `NULL`, ce qui crée des surprises entre moteurs. Tes contrôles de complétude deviennent faux.

Correction : Filtre les deux cas : `WHERE email IS NOT NULL AND email <> ''`. Ou normalise en amont avec `NULLIF(email, '')` pour ramener le vide à `NULL`.

15. Le tri qui place les `NULL` là où tu ne les attends pas

Le piège : `ORDER BY date_resiliation ASC` pour lister les départs du plus ancien au plus récent, sans penser aux `NULL`.

Symptôme : Les clients actifs (`NULL` = pas résilié) remontent ou descendent selon le moteur, brouillant ta lecture.

Conséquence : Le placement des `NULL` dans un tri n'est pas universel : en `ASC`, PostgreSQL met les `NULL` en dernier, MySQL en premier (et l'inverse en `DESC`). Un `top N` « par date » peut renvoyer les mauvaises lignes.

Correction : Sois explicite avec `NULLS FIRST / NULLS LAST` (PostgreSQL, Oracle, BigQuery). Sur MySQL, simule avec `ORDER BY col IS NULL, col`.

■ ■ ■ Fenêtrage (window functions) : garder le détail tout en calculant par groupe

16. Filtrer une window function dans le `WHERE`

Le piège : `WHERE ROW_NUMBER() OVER (...) = 1` pour ne garder que la première ligne par groupe.

Symptôme : Erreur « window function is not allowed in WHERE ».

Conséquence : Les window functions sont évaluées au moment du `SELECT`, donc après le `WHERE`. Tu ne peux pas filtrer sur un rang qui n'est pas encore calculé.

Correction : Calcule le rang dans une CTE, puis filtre dans la requête externe. Sur BigQuery, la clause `QUALIFY` permet de filtrer après les window functions sans CTE.

```
WITH r AS (  
  SELECT *, ROW_NUMBER() OVER (  
    PARTITION BY client_id ORDER BY date_cmd  
  ) AS rn  
  FROM commandes  
)  
SELECT * FROM r WHERE rn = 1;
```

17. Choisir `RANK` là où il faut `ROW_NUMBER` (ou l'inverse)

Le piège : Tu veux dédoubler et tu utilises RANK, ou tu veux un classement métier et tu utilises ROW_NUMBER.

Symptôme : Sur des ex-aequo (10, 10, 20) : ROW_NUMBER donne 1, 2, 3; RANK donne 1, 1, 3; DENSE_RANK donne 1, 1, 2.

Conséquence : Avec RANK pour dédoubler, tu gardes deux lignes au rang 1. Avec ROW_NUMBER pour un podium, tu départages arbitrairement deux égalités réelles.

Correction : ROW_NUMBER pour garder une seule ligne par groupe, RANK pour un classement avec trous, DENSE_RANK pour un classement sans trou. Et un ORDER BY déterministe (ajoute une clé unique) pour rendre ROW_NUMBER reproductible.

18. La moyenne mobile « 7 lignes » prise pour « 7 jours »

Le piège : AVG(...) OVER (ORDER BY jour ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) sur des données à trous.

Symptôme : La fenêtre couvre 7 lignes, donc parfois bien plus de 7 jours si des dates manquent.

Conséquence : Ta « moyenne 7 jours » mélange des périodes de longueurs différentes selon les trous. Les variations de trafic se lissent ou s'amplifient de façon incohérente.

Correction : Joins d'abord tes données sur un calendrier complet (spine de dates) pour garantir une ligne par jour. Ou utilise RANGE BETWEEN INTERVAL '6 days' PRECEDING AND CURRENT ROW, qui raisonne sur les valeurs de date et non sur les lignes.

19. Oublier la frame par défaut avec un ORDER BY dans le OVER

Le piège : SUM(montant) OVER (PARTITION BY client ORDER BY date) en croyant obtenir le total du client.

Symptôme : Tu obtiens un cumul progressif (running total), pas un total fixe.

Conséquence : Dès qu'un ORDER BY est présent dans le OVER, la frame par défaut devient RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. Sans ORDER BY, c'est tout le partition. Tu confonds cumul et total.

Correction : Pour un total par groupe, retire le ORDER BY du OVER. Pour un cumul, garde-le mais sache que c'est un cumul. Et méfie-toi de RANGE sur des ex-aequo de tri : il agrège tous les pairs, pas la seule ligne courante (préfère ROWS si tu veux un vrai cumul ligne à ligne).

■ ■ ■ Sous-requêtes et CTE : structurer sans se piéger

20. La sous-requête corrélée qui s'exécute ligne par ligne

Le piège : Une sous-requête dans le SELECT qui référence la table externe, exécutée pour chaque ligne.

Symptôme : La requête est correcte mais devient très lente quand le volume grandit.

Conséquence : Une corrélation mal placée peut multiplier les passes sur la table. Sur de gros volumes, le temps explose là où une jointure ferait le travail en une passe.

Correction : Remplace la sous-requête corrélée par un JOIN sur une agrégation, ou par une window function. Les optimiseurs modernes décorrèlent parfois, mais ne compte pas dessus : vérifie le plan d'exécution.

21. Une sous-requête IN qui renvoie des NULL

Le piège : WHERE client_id IN (SELECT ...) où la sous-requête peut produire des NULL.

Symptôme : Avec IN, les NULL sont au mieux ignorés; avec NOT IN, ils cassent tout le résultat (voir piège 4).

Conséquence : Le comportement face aux NULL d'une liste IN/NOT IN est rarement celui qu'on imagine, et le bug ne se voit pas à la lecture.

Correction : Préfère EXISTS / NOT EXISTS, qui raisonnent sur la présence de lignes et restent fiables face aux NULL. Et n'expose jamais une colonne nullable dans une sous-requête de NOT IN.

22. Croire qu'une CTE est toujours gratuite et optimisée

Le piège : Tu découpes en CTE pour la lisibilité, en supposant que le moteur les fond comme des sous-requêtes.

Symptôme : Sur certaines versions, une CTE lue plusieurs fois est recalculée, ou matérialisée et plus lente.

Conséquence : Sur PostgreSQL avant la v12, une CTE est matérialisée par défaut : l'optimiseur ne peut pas pousser les filtres à travers, ce qui freine. Le code lisible n'est pas toujours le code rapide.

Correction : Garde les CTE pour la clarté (c'est souvent le bon choix), mais vérifie le plan d'exécution sur les requêtes critiques. Sur PostgreSQL récent, tu peux forcer avec MATERIALIZED / NOT MATERIALIZED.

23. La CTE récursive sans condition d'arrêt

Le piège : Une WITH RECURSIVE pour parcourir une hiérarchie (manager, arborescence) sans garde-fou.

Symptôme : La requête tourne sans fin, ou explose la mémoire, surtout si les données contiennent un cycle.

Conséquence : Une boucle dans les données (A manager de B, B manager de A) fait diverger la récursion à l'infini. En production, c'est une requête qui sature le serveur.

Correction : Borne la profondeur avec un compteur de niveau dans la partie récursive,

et détecte les cycles (par exemple en accumulant le chemin parcouru). Teste toujours sur un échantillon avec une limite de niveaux.

■ ■ ■ Types et dates : les conversions qui mentent

24. La division entière prise pour une division décimale

Le piège : `taux = succes / total` avec deux colonnes entières.

Symptôme : Le ratio vaut 0 (ou un entier tronqué), jamais 0.42.

Conséquence : Sur PostgreSQL et SQL Server, entier divisé par entier donne un entier tronqué. Un taux de conversion qui affiche 0 % partout est un grand classique de junior.

Correction : Force le décimal : `succes * 1.0 / total` ou `CAST(succes AS DECIMAL) / total`. Et protège la division par zéro avec `NULLIF(total, 0)` au dénominateur.

25. CAST brutal sur des données sales

Le piège : `CAST(colonne AS INT)` sur un import CSV où une valeur contient du texte.

Symptôme : La requête plante sur toute la ligne d'import dès la première valeur non convertible.

Conséquence : Un seul enregistrement corrompu bloque tout le traitement. En pipeline, c'est une nuit de chargement qui échoue pour une cellule.

Correction : Utilise la variante « safe » : `TRY_CAST` (SQL Server), `SAFE_CAST` (BigQuery), qui renvoient `NULL` au lieu de planter. Sur PostgreSQL, pas d'équivalent natif : passe par un `CASE` avec une regex de validation.

26. Comparer un TIMESTAMP à une date du jour

Le piège : `WHERE date_commande = '2026-06-22'` sur une colonne `TIMESTAMP`.

Symptôme : Tu rates toutes les commandes du jour passées après minuit, car `2026-06-22 14:30` n'égal pas `2026-06-22 00:00`.

Conséquence : La comparaison d'égalité sur un timestamp ne matche que minuit pile. Ton « ventes du jour » renvoie presque toujours zéro.

Correction : Utilise une plage demi-ouverte : `WHERE date_commande >= '2026-06-22' AND date_commande < '2026-06-23'`. Évite `BETWEEN` sur des timestamps : il inclut la borne haute et rate la dernière journée si tu bornes à minuit.

27. Les fuseaux horaires stockés en local

Le piège : Tu stockes les dates dans le fuseau du serveur ou de l'application.

Symptôme : Les clôtures mensuelles et les cohortes changent selon la machine qui

exécute la requête.

Conséquence : Une transaction de 23h59 à Paris peut basculer de jour, voire de mois, vue depuis un serveur en UTC. Tes métriques temporelles deviennent non reproductibles.

Correction : Stocke tout en UTC, convertis vers le fuseau local seulement à l'affichage. Utilise `TIMESTAMPTZ` (PostgreSQL) ou un type équivalent, et documente le fuseau de référence du reporting.

■ ■ ■ Performance : la requête juste mais lente

28. UNION par défaut au lieu de UNION ALL

Le piège : UNION pour empiler deux résultats dont tu sais déjà qu'ils ne se recoupent pas.

Symptôme : La requête est plus lente que nécessaire, sans raison apparente.

Conséquence : UNION dédouble, ce qui impose un tri ou un hash coûteux sur l'ensemble. Sur de gros volumes, c'est un surcoût pur si la déduplication est inutile.

Correction : Utilise UNION ALL par défaut. Ne garde UNION que si tu as un vrai besoin de dédoublement, et que tu acceptes son coût.

29. Une fonction sur la colonne dans le WHERE qui tue l'index

Le piège : WHERE YEAR(date_commande) = 2026 OU WHERE UPPER(email) = '...'.

Symptôme : La requête fait un Full Table Scan alors qu'un index existe sur la colonne.

Conséquence : Appliquer une fonction à la colonne la rend « non sargable » : l'index sur la colonne brute ne peut plus servir, et le moteur scanne toute la table.

Correction : Réécris pour laisser la colonne nue : WHERE date_commande >= '2026-01-01' AND date_commande < '2027-01-01'. Sinon, crée un index sur l'expression (index fonctionnel) quand le moteur le permet.

30. LIKE '%texte%' avec un joker en tête

Le piège : WHERE nom LIKE '%arnaud%' en supposant que l'index s'en charge.

Symptôme : La recherche est lente et se dégrade avec le volume.

Conséquence : Un joker % en début de pattern empêche un index B-Tree de servir (il indexe les préfixes). Résultat : Full Table Scan à chaque recherche.

Correction : Si tu peux ancrer le début (LIKE 'arnaud%'), l'index B-Tree fonctionne. Pour de la recherche « contient », passe à un index Full-Text ou trigramme (pg_trgm sur PostgreSQL, qui indexe même les LIKE/ILIKE non ancrés).